Reviewer UX Brief for AI-Assisted Legacy Modernization

A condensed synthesis of architecture-description standards, requirements engineering guidance, human-AI interaction guidelines, explainability guidance, and software inspection practice. The objective is practical: make a legacy-system review process usable with sparse documentation, limited reviewers, and low tolerance for cognitive overload.

Scope: reviewer experience for generated modernization artifacts Target length: 5 pages max Prepared: April 11, 2026

Bottom line
The best review process is not architecture-first. It is **capability-based, requirements-centered, and exception-driven**. Architecture views help reviewers orient themselves, but the main artifact for approval should be a short, evidence-backed business requirements pack.

What The Literature Converges On

1. Different reviewers need different views

- Architecture descriptions should be built around stakeholder concerns and viewpoints, not one universal document.
- One dense diagram is usually the wrong answer for mixed technical and business audiences.
- Use only the views that answer an actual review question.

2. Requirements are easier to review than structure

- Reviewers are better at validating obligations, workflows, exceptions, and outcomes than large dependency graphs.
- Generated requirements should be clear, bounded, necessary, and traceable.
- Architecture should support those requirements, not compete with them.

3. Explain for decisions, not completeness

- Human-AI guidance consistently favors progressive disclosure over dumping full system reasoning into the main flow.
- Default reviewers need concise context, the proposed interpretation, uncertainty, and an easy correction path.
- Evidence should be one click away, not always-on.

4. Limited reviewer time is a design constraint

- Formal review literature treats reviewer attention as scarce and defect-oriented.
- The process should surface high-risk, low-confidence, and high-impact items first.
- Everything else should be batchable or spot-checkable.

Implication for your use case
If the generated output starts with a giant source architecture document, reviewers will burn time understanding the artifact itself. If it starts with a capability summary and 5-12 requirement cards, they can spend time validating the system.

Recommended Review Hierarchy

Layer	Primary question	Audience	Default visibility
Business requirements	What must the system do?	Business reviewers, architects, senior engineers	Always visible
Workflow / sequence view	How does a critical journey unfold?	Mixed reviewers	Visible when needed
Architecture context	Where does this capability live and what does it touch?	Architects, engineers	Visible as supporting context
Evidence / traceability	Why does the system think this requirement exists?	Analysts, engineers, SMEs	On demand only

Recommended Review Process

Review should happen in **small capability slices**, not as a whole-system documentation exercise. A capability slice is something like *order submission*, *customer onboarding*, or *report distribution*, rather than a code module or file tree.

Stage	What happens	Why this is practical
1. Machine draft	Generate draft requirements, one workflow view, one context view, and an evidence bundle for each capability.	Keeps the AI output bounded and review-shaped.
2. Analyst scrub	A technical analyst removes obvious duplicates, fixes naming, groups related requirements, and marks likely low-value noise.	Protects scarce reviewers from spending time on cleanup.
3. Priority scoring	Score each item by business impact, confidence, change risk, and reviewer uncertainty.	Lets the process be exception-driven rather than exhaustive.
4. Live review	Review only the highest-value items in a 30-45 minute session, capability by capability.	Fits the reality of limited reviewer availability.
5. Structured resolution	Every item ends as `confirmed`, `corrected`, `unclear`, or `deferred`.	Avoids comments with no decision outcome.
6. Incremental lock	Freeze approved capability slices instead of waiting for whole-estate completeness.	Reduces coordination cost and allows progress despite legacy uncertainty.

Session Design For Limited Reviewers

Before the meeting

- Send one capability pack only.
- Highlight top 3 unresolved items.
- Pre-label low-confidence interpretations.
- Keep the pre-read under 10 minutes.

During the meeting

- Start with business outcome, not the diagram.
- Review the main flow first, exceptions second.
- Escalate to evidence only when challenged.
- Capture decisions live in structured form.

After the meeting

- Regenerate only changed requirement cards.
- Send unresolved questions to SMEs only.
- Track drift between approved requirements and later architecture/code proposals.

Default review thresholds

- 5-12 substantive requirement cards per pack.
- 30-45 minutes per live session.
- No more than 2 diagrams by default.
- Only 1 capability in active review at a time per reviewer group.

Practical rule
Reviewers should never need to understand the full architecture to validate a capability. If they do, the review pack is too dense or sliced at the wrong level.

What Documentation Should Be Generated

Generate documentation in layers, with each layer optimized for a different decision. The key is not “more documentation”; it is **purpose-specific documentation**.

Artifact	Purpose	Main audience	Required content
Review index	Portfolio view of all capabilities and review status	Leads, coordinators	Capability name, owner, status, confidence, top risk, next action
Capability review pack	Main approval artifact	All reviewers	Business summary, 5-12 requirement cards, top exceptions, top open questions
Workflow / sequence view	Clarify main and exception paths	Mixed reviewers	Actors, trigger, path, exception branch, key data/state handoffs
Context / architecture view	Orient the reviewer structurally	Architects, engineers	Subsystems, external systems, shared stores, hotspots
Evidence appendix	Justify inferred requirements	Analysts, SMEs	Source paths, facts, queries, config references, confidence rationale
Decision log	Preserve corrections and rationale	All downstream users	Decision, reviewer, date, reason, impacted requirement(s)
Open questions register	Manage unresolved uncertainty	SMEs, coordinators	Question, impact, owner, deadline, related capability

What A Capability Review Pack Should Contain

Page 1: Business summary

- Capability name and why it exists
- Primary actors
- Business outcome
- Main systems/data involved
- Top 3 known risks or unknowns

Page 2: Requirement cards

- Trigger
- Conditions / preconditions
- Expected outcome
- Exceptions
- Confidence and evidence link

Optional support

- One workflow / sequence diagram
- One simple context diagram
- Only include hotspot diagrams when they explain a specific risk

Decision footer

- Confirmed items
- Corrected items
- Deferred items
- Questions sent to SME

Do not generate as default review artifacts
One giant source architecture document, full dependency graph exports, raw fact dumps, raw AI prose, or a monolithic BRD for the whole estate.

Reviewer UX Rules To Minimize Cognitive Overload

Rule	Why it matters	Practical implementation
Start with the task, not the model	Reviewers care about validating business behavior, not model internals.	Default screen opens with summary + requirement cards.
Use progressive disclosure	Human-AI guidance warns against showing full explanation up front.	Hide evidence, source mappings, and low-level details behind expandable sections.
Show uncertainty explicitly	Trust improves when ambiguity is visible rather than implied away.	Label items as inferred, low confidence, or unresolved.
Make correction cheap	Reviewer effort should go into decisions, not navigation or rewriting.	Provide structured edits: rename, change condition, add exception, mark unknown.
Keep diagrams sparse and purposeful	Diagrams help only when they answer a specific concern.	Use one context view and one dynamic/workflow view by default.
Separate review from investigation	Mixing both creates overload and derails sessions.	Move deep evidence walks and parser disputes into offline follow-up.

Suggested Decision States

Requirement state

- `draft`
- `needs-review`
- `confirmed`
- `corrected`
- `deferred`
- `locked`

Question state

- `open`
- `needs-sme`
- `resolved`
- `accepted-risk`

Metrics Worth Tracking

- Average review time per capability
- Percentage of draft requirements confirmed without edit
- Percentage requiring correction
- Number of unresolved SME escalations
- Time from first draft to locked review pack
- Drift between locked requirements and later target design / verification failures

> **Success criterion**
> A reviewer should be able to answer “is this capability understood well enough to modernize?” within one short session, without reading a system-wide architecture document.

Adoption Guidance And Source Trail

Recommended operating model

1. Generate review packs by capability.
2. Have one analyst scrub each pack before stakeholder review.
3. Make business requirements the main approval artifact.
4. Keep architecture views as orientation artifacts.
5. Use structured decisions and incremental locks.
6. Escalate only unresolved, high-impact questions to scarce SMEs.

Source list

- ISO/IEC/IEEE 42010:2022, architecture description and stakeholder viewpoints: https://www.iso.org/standard/74393.html
- ISO/IEC/IEEE 29148:2018, requirements engineering lifecycle processes: https://www.iso.org/standard/72089.html
- C4 Model introduction and diagram guidance: https://c4model.com/introduction and https://c4model.com/diagrams
- Microsoft Guidelines for Human-AI Interaction / HAX Toolkit: https://www.microsoft.com/en-us/research/publication/guidelines-for-human-ai-interaction/ and https://www.microsoft.com/en-us/haxtoolkit/
- Google PAIR Guidebook patterns for human-centered AI UX: https://pair.withgoogle.com/guidebook-v2/patterns
- NIST, Four Principles of Explainable Artificial Intelligence: https://www.nist.gov/publications/four-principles-explainable-artificial-intelligence
- NIST, AI Use Taxonomy: A Human-Centered Approach: https://www.nist.gov/publications/ai-use-taxonomy-human-centered-approach
- NASA-STD-8739.9, Software Formal Inspections Standard: https://ntrs.nasa.gov/citations/20180004877
- ONS Service Manual, diagrams guidance: https://service-manual.ons.gov.uk/content/content-types/diagrams
- SEBoK discussion of views and viewpoints as stakeholder concern mechanisms: https://sebokwiki.org/wiki/Guide_to_the_Systems_Engineering_Body_of_Knowledge_(SEBoK)

This brief is a synthesis, not a standard. It intentionally translates the source material into one practical recommendation for AI-assisted modernization reviews: short capability packs, progressive disclosure, structured reviewer decisions, and evidence on demand.